Reinforcement Learning Laboratory

Lab Experiment 1: Exploring Reinforcement Learning Environments using Gymnasium

Course: Reinforcement Learning (22AIE401) **Course Outcome:** CO1 — Define the key features of reinforcement learning that distinguish it from AI and non-interactive machine learning **Duration:** 3 Hours

Objectives

1. Install the Reinforcement Learning development environment.
2. Create and execute Gymnasium environments.
3. Explore the observation and action spaces of RL environments.
4. Execute a random agent.
5. Observe rewards, episode termination, and environment behaviour.
6. Compare different RL benchmark environments.

1. Setting up the Development Environment

`Gymnasium` (the actively maintained fork of OpenAI Gym) is the standard toolkit for developing and comparing RL algorithms. It provides a unified API (`reset` , `step` , `render` , `close`) across a wide variety of benchmark environments.

Run the cell below once to install the required packages (`gymnasium` , `numpy` , `matplotlib`). In Google Colab, prefix with `!` ; in a local terminal drop the `!` .

In [1]:
```
# Run this once to set up the environment
!pip install gymnasium numpy matplotlib -q
print("Setup complete.")
```

```
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    If you wish to install a non-Debian packaged Python application,
    it may be easiest to use pipx install xyz, which will manage a
    virtual environment for you. Make sure you have pipx installed.

    See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or
OS distribution provider. You can override this, at the risk of breaking your Pytho
n installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
Setup complete.
```

In [2]:
```python
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

print("Gymnasium version:", gym.__version__)
```

```
Gymnasium version: 1.3.0
```

2. Creating and Executing a Gymnasium Environment

Every Gymnasium environment follows the same lifecycle:

Step	Method	Purpose
Create	`gym.make(env_id)`	Instantiate the environment
Reset	`env.reset()`	Start a new episode, returns initial observation
Step	`env.step(action)`	Apply an action, returns `(observation, reward, terminated, truncated, info)`
Close	`env.close()`	Release environment resources

We start with **CartPole-v1**, a classic control task where the goal is to balance a pole on a moving cart.

In [3]:
```python
env = gym.make("CartPole-v1")

obs, info = env.reset(seed=42)
print("Initial observation:", obs)
print("Info:", info)

# Take a single random step to see the API in action
```

```
action = env.action_space.sample()
obs, reward, terminated, truncated, info = env.step(action)
print("\nAfter one step:")
print("Action taken:", action)
print("New observation:", obs)
print("Reward:", reward)
print("Terminated:", terminated, "| Truncated:", truncated)

env.close()
```

```
Initial observation: [ 0.0273956  -0.00611216  0.03585979  0.0197368 ]
Info: {}

After one step:
Action taken: 0
New observation: [ 0.02727336 -0.20172954  0.03625453  0.32351476]
Reward: 1.0
Terminated: False | Truncated: False
```

3. Exploring Observation and Action Spaces

The **observation space** defines the set of all possible states the agent can perceive. The **action space** defines the set of all possible actions the agent can take. Gymnasium describes both using `gym.spaces` objects such as `Discrete` (a finite set of integers) and `Box` (a continuous, bounded range of real values).

We inspect these spaces for four benchmark environments spanning different characteristics:

- **CartPole-v1** — continuous observation space, discrete action space
- **MountainCar-v0** — continuous observation space, discrete action space, sparse/delayed reward
- **FrozenLake-v1** — discrete observation space, discrete action space, stochastic transitions
- **Taxi-v4** — discrete observation space, discrete action space, composite reward structure

In [4]:
```
env_names = ["CartPole-v1", "MountainCar-v0", "FrozenLake-v1", "Taxi-v4"]
env_info = {}

for name in env_names:
    env = gym.make(name)
    env_info[name] = {
        "obs_space": env.observation_space,
        "action_space": env.action_space,
        "max_episode_steps": env.spec.max_episode_steps,
    }
    print(f"=== {name} ===")
    print("Observation space:", env.observation_space)
    print("Action space:", env.action_space)
    print("Max episode steps:", env.spec.max_episode_steps)
    print()
    env.close()
```

```
=== CartPole-v1 ===
Observation space: Box([-4.8               -inf -0.41887903        -inf], [4.8
inf 0.41887903        inf], (4,), float32)
Action space: Discrete(2)
Max episode steps: 500

=== MountainCar-v0 ===
Observation space: Box([-1.2  -0.07], [0.6  0.07], (2,), float32)
Action space: Discrete(3)
Max episode steps: 200

=== FrozenLake-v1 ===
Observation space: Discrete(16)
Action space: Discrete(4)
Max episode steps: 100

=== Taxi-v4 ===
Observation space: Discrete(500)
Action space: Discrete(6)
Max episode steps: 200
```

Observations

- `CartPole-v1` 's observation space is a `Box` of 4 continuous values: cart position, cart velocity, pole angle, and pole angular velocity. Its action space is `Discrete(2)` — push cart left or right.
- `MountainCar-v0` 's observation space is a `Box` of 2 continuous values: car position and velocity. Its action space is `Discrete(3)` — push left, do nothing, push right.
- `FrozenLake-v1` 's observation space is `Discrete(16)` — the agent's cell index on a 4×4 grid. Its action space is `Discrete(4)` — move up, down, left, right.
- `Taxi-v4` 's observation space is `Discrete(500)` , encoding taxi position, passenger location, and destination. Its action space is `Discrete(6)` — 4 movement directions plus pickup/drop-off.

This shows how Gymnasium environments differ in **state representation** (continuous vs. discrete) even though the underlying `step` / `reset` API stays identical — a key design idea behind the RL environment abstraction.

4. Executing a Random Agent

A **random agent** selects actions uniformly at random from the action space at every timestep, without learning. It serves as a baseline: any real RL algorithm should eventually outperform it. Running a random agent also lets us observe the raw reward and termination signals an environment produces before any learning takes place.

In [5]:
```python
def run_random_agent(env_name, episodes=5, seed=42, verbose=True):
    env = gym.make(env_name)
    episode_data = []

    for ep in range(episodes):
        obs, info = env.reset(seed=seed + ep)
```

```
        total_reward = 0.0
        steps = 0
        terminated = truncated = False

        while not (terminated or truncated):
            action = env.action_space.sample()
            obs, reward, terminated, truncated, info = env.step(action)
            total_reward += reward
            steps += 1

        episode_data.append({
            "episode": ep + 1,
            "reward": total_reward,
            "steps": steps,
            "terminated": terminated,
            "truncated": truncated,
        })
        if verbose:
            print(f"Episode {ep+1}: Reward={total_reward:.1f}, Steps={steps}, "
                  f"Terminated={terminated}, Truncated={truncated}")

    env.close()
    return episode_data

print("Random Agent on CartPole-v1")
cartpole_results = run_random_agent("CartPole-v1")
```

```
Random Agent on CartPole-v1
Episode 1: Reward=32.0, Steps=32, Terminated=True, Truncated=False
Episode 2: Reward=46.0, Steps=46, Terminated=True, Truncated=False
Episode 3: Reward=22.0, Steps=22, Terminated=True, Truncated=False
Episode 4: Reward=15.0, Steps=15, Terminated=True, Truncated=False
Episode 5: Reward=24.0, Steps=24, Terminated=True, Truncated=False
```

5. Observing Rewards, Episode Termination, and Environment Behaviour

We now run the same random-agent procedure across all four environments and compare how reward signals and termination conditions differ.

- **`terminated=True`** means the episode ended due to the environment's own success/failure condition (e.g. pole fell, taxi delivered passenger, agent fell in a hole).
- **`truncated=True`** means the episode was cut off by a time limit (`max_episode_steps`), not by the environment's internal logic.

In [6]:
```
all_results = {}
for name in env_names:
    print(f"\n--- {name} ---")
    all_results[name] = run_random_agent(name)
```

```
--- CartPole-v1 ---
Episode 1: Reward=16.0, Steps=16, Terminated=True, Truncated=False
Episode 2: Reward=32.0, Steps=32, Terminated=True, Truncated=False
Episode 3: Reward=25.0, Steps=25, Terminated=True, Truncated=False
Episode 4: Reward=49.0, Steps=49, Terminated=True, Truncated=False
Episode 5: Reward=36.0, Steps=36, Terminated=True, Truncated=False

--- MountainCar-v0 ---
Episode 1: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 2: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 3: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 4: Reward=-200.0, Steps=200, Terminated=False, Truncated=True
Episode 5: Reward=-200.0, Steps=200, Terminated=False, Truncated=True

--- FrozenLake-v1 ---
Episode 1: Reward=0.0, Steps=3, Terminated=True, Truncated=False
Episode 2: Reward=0.0, Steps=12, Terminated=True, Truncated=False
Episode 3: Reward=0.0, Steps=7, Terminated=True, Truncated=False
Episode 4: Reward=0.0, Steps=11, Terminated=True, Truncated=False
Episode 5: Reward=0.0, Steps=3, Terminated=True, Truncated=False

--- Taxi-v4 ---
Episode 1: Reward=-695.0, Steps=200, Terminated=False, Truncated=True
Episode 2: Reward=-758.0, Steps=200, Terminated=False, Truncated=True
Episode 3: Reward=-794.0, Steps=200, Terminated=False, Truncated=True
Episode 4: Reward=-740.0, Steps=200, Terminated=False, Truncated=True
Episode 5: Reward=-830.0, Steps=200, Terminated=False, Truncated=True
```

In [7]:
```python
summary_rows = []
for name, episodes in all_results.items():
    rewards = [e["reward"] for e in episodes]
    steps = [e["steps"] for e in episodes]
    term_rate = np.mean([e["terminated"] for e in episodes])
    summary_rows.append({
        "env": name,
        "avg_reward": np.mean(rewards),
        "std_reward": np.std(rewards),
        "avg_steps": np.mean(steps),
        "terminated_rate": term_rate,
    })

print(f"{'Environment':<16}{'AvgReward':>12}{'StdReward':>12}{'AvgSteps':>10}{'Ter
for row in summary_rows:
    print(f"{row['env']:<16}{row['avg_reward']:>12.2f}{row['std_reward']:>12.2f}"
          f"{row['avg_steps']:>10.1f}{row['terminated_rate']:>10.2f}")
```

```
Environment        AvgReward   StdReward  AvgSteps  TermRate
CartPole-v1            31.60       11.04      31.6      1.00
MountainCar-v0       -200.00        0.00     200.0      0.00
FrozenLake-v1           0.00        0.00       7.2      1.00
Taxi-v4              -763.40       46.10     200.0      0.00
```

Observations on Reward and Termination Behaviour

- **CartPole-v1**: Reward is `+1` for every timestep the pole stays upright, so total reward equals the number of steps survived. A random agent survives only briefly (well under the 500-step cap) before the pole falls and the episode **terminates**.

- **MountainCar-v0**: Reward is `-1` per timestep until the car reaches the goal flag. A random agent almost never has enough directed momentum to reach the goal, so every episode runs to the 200-step limit and is **truncated** with total reward `-200`. This illustrates the *sparse/delayed reward* problem in RL.
- **FrozenLake-v1**: Reward is `0` for every step and `+1` only on reaching the goal without falling in a hole. Because the ice is slippery (stochastic transitions), a random agent almost always falls into a hole quickly, so reward stays at `0` and the episode **terminates** early.
- **Taxi-v4**: Reward is `-1` per timestep, `-10` for illegal pickup/drop-off, and `+20` for a successful drop-off. A random agent racks up large negative reward from illegal actions and rarely completes deliveries within the step limit, so most episodes are **truncated** with very negative reward.

Together these show how **reward density** (dense vs. sparse), **stochasticity**, and **termination conditions** vary across benchmark tasks, and why a random baseline is far from optimal in every case.

6. Comparing RL Benchmark Environments

The table and chart below summarise the structural and empirical differences between the four environments explored in this lab.

In [8]:
```
import pandas as pd

comparison = pd.DataFrame([
    {
        "Environment": "CartPole-v1",
        "Obs Space": "Box(4,) continuous",
        "Action Space": "Discrete(2)",
        "Reward Type": "Dense (+1/step)",
        "Stochastic?": "No",
        "Max Steps": 500,
    },
    {
        "Environment": "MountainCar-v0",
        "Obs Space": "Box(2,) continuous",
        "Action Space": "Discrete(3)",
        "Reward Type": "Dense but sparse goal (-1/step)",
        "Stochastic?": "No",
        "Max Steps": 200,
    },
    {
        "Environment": "FrozenLake-v1",
        "Obs Space": "Discrete(16)",
        "Action Space": "Discrete(4)",
        "Reward Type": "Sparse (0 / +1 at goal)",
        "Stochastic?": "Yes (slippery)",
        "Max Steps": 100,
    },
    {
        "Environment": "Taxi-v4",
        "Obs Space": "Discrete(500)",
        "Action Space": "Discrete(6)",
```

```
        "Reward Type": "Dense w/ penalties (-1, -10, +20)",
        "Stochastic?": "No",
        "Max Steps": 200,
    },
])
comparison
```

Out[8]:

	Environment	Obs Space	Action Space	Reward Type	Stochastic?	Max Steps
0	CartPole-v1	Box(4,) continuous	Discrete(2)	Dense (+1/step)	No	500
1	MountainCar-v0	Box(2,) continuous	Discrete(3)	Dense but sparse goal (-1/step)	No	200
2	FrozenLake-v1	Discrete(16)	Discrete(4)	Sparse (0 / +1 at goal)	Yes (slippery)	100
3	Taxi-v4	Discrete(500)	Discrete(6)	Dense w/ penalties (-1, -10, +20)	No	200

In [9]:

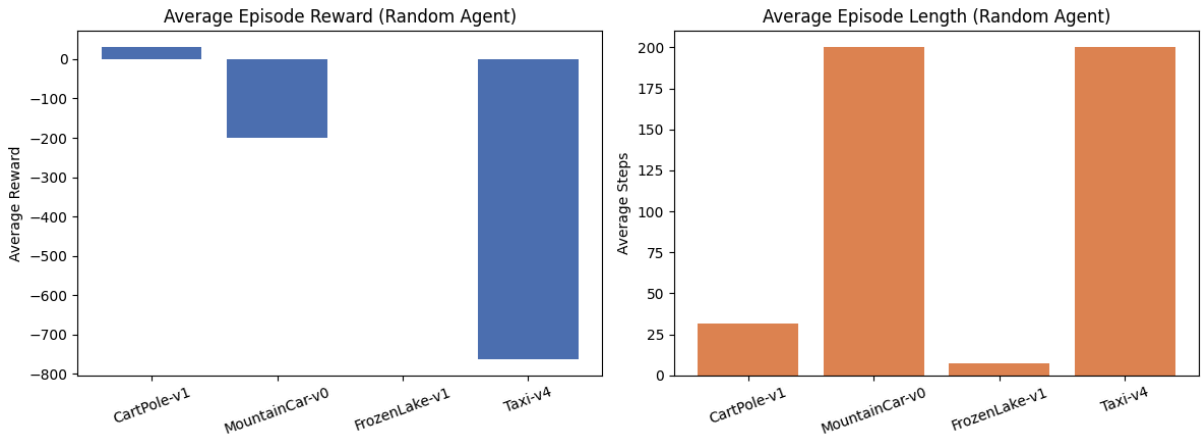
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

names = [row["env"] for row in summary_rows]
avg_rewards = [row["avg_reward"] for row in summary_rows]
avg_steps = [row["avg_steps"] for row in summary_rows]

axes[0].bar(names, avg_rewards, color="#4C72B0")
axes[0].set_title("Average Episode Reward (Random Agent)")
axes[0].set_ylabel("Average Reward")
axes[0].tick_params(axis='x', rotation=20)

axes[1].bar(names, avg_steps, color="#DD8452")
axes[1].set_title("Average Episode Length (Random Agent)")
axes[1].set_ylabel("Average Steps")
axes[1].tick_params(axis='x', rotation=20)

plt.tight_layout()
plt.savefig("random_agent_comparison.png", dpi=150)
plt.show()
```



Discussion

- Environments with **continuous state spaces** (CartPole, MountainCar) need function approximation (e.g. neural networks) once we move beyond tabular methods, since the state space is infinite.
- Environments with **discrete, finite state spaces** (FrozenLake, Taxi) are well suited to *tabular* RL methods (e.g. Q-tables), which is why they appear early in the syllabus alongside Dynamic Programming and Monte Carlo methods.
- **Stochastic transitions** (FrozenLake's slippery ice) make an environment harder to solve than a deterministic one with the same size, since the agent must learn a policy that is robust to unpredictable outcomes.
- **Sparse rewards** (MountainCar, FrozenLake) make random exploration highly inefficient — this motivates the exploration-vs-exploitation techniques (e.g. ε-greedy, UCB) covered later in the course.

7. Conclusion

This lab introduced the Gymnasium API and demonstrated:

- How to install and set up an RL development environment.
- How to create, reset, and step through environments using a uniform API.
- How observation and action spaces differ across environments (`Box` vs. `Discrete`).
- How a random agent behaves as a baseline, and how reward/termination signals reveal an environment's underlying difficulty.
- How environments can be systematically compared along axes such as state representation, reward density, and stochasticity.

These comparisons directly motivate the algorithms studied later in the course (Dynamic Programming, Monte Carlo, Temporal-Difference Learning), each suited to different combinations of these environment properties.